Online Resource 5

Reconstruction attempt for the predictor of Halle et al. (2021)

Supplementary material to: *How the Classical Baseline Shapes Machine-Learning Speed Claims in Topology Optimization.* Shaoliang Yang and Jun Wang, Santa Clara University. Corresponding author: Jun Wang, jwang22@scu.edu. Computers & Structures.

Reading guide. Compliance measures deformation under load; lower compliance means a stiffer design under the same conditions. The 88-line classical topology-optimization code is called top88. CPU/GPU, central/graphics processing unit; OC, optimality criteria; S-SuperLU and S-PARDISO denote the reference and alternative CPU solvers, and S-GPU the GPU solver. The stored reference designs and the attempted reconstruction are distinct: the failed check assesses this reconstruction and prevents a timing comparison. fem2d is our two-dimensional finite-element model, the single evaluator used here; P00150 is the audit record number of Halle et al. (2021). The baseline pre-registration and its dated rows (§9) are reproduced in Online Resource 1. In these documents, pre-registered means fixed in writing before the coding or runs it governs, in a hashed plan or a dated log row entered before the run; the hashes are held in the authors' records, without an external registry timestamp.

Halle et al. (2021) released a trained predictor together with stored top88 designs and their compliances. The pre-registered re-test required a check before any timing: our top88 port, run to its stopping point on the same 100 cases, had to reproduce the stored compliance within 1 % for at least 95 of them. The check failed twice, so no speed comparison for this article is reported in the paper. This resource gives the pre-registered entries, the diagnosis with its two corrections, the final result and the scripts. The entries are reproduced from the baseline pre-registration; a local software path is replaced by a description.

1. Pre-registered entries and the diagnosis (baseline pre-registration, §9)

Date	Section	Change	Reason
2026-09-24	P00150 re-test (Halle et al. 2021; §7), recorded before any P00150 run	(1) **Target:** the article's trained predictor `model_predictor_64x64` and its 100 validation cases with the stored top88 results (Mendeley data, CC BY 4.0; `retest/targets/P00150_mendeley/`, SHA-256 in `DOWNLOAD_MANIFEST.md`), used unmodified. The predictor runs in a new conda env `tf_p00150` (Python 3.11, TensorFlow 2.15, CPU only, because TensorFlow after 2.10 has no GPU support on native Windows). (2) **Problems:** each validation input decodes to a 64×64 problem: left edge clamped, one unit point load at a node of the 8×8 input grid, and a target volume fraction (`retest/p00150_harness/problems.py`). Settings follow the article's Table 2: p = 3, r_min = 3, $\nu = 0.3$. Compliance is computed with E = 1 and a unit load, then multiplied by $100^2/195{,}000$ to give the article's units. (3) **Reconstruction check before any comparison:** our top88 port (S-SuperLU path) must reproduce the stored top88 compliance within 1 % at top88's stopping point (design change ≤ 0.01) on at least 95 of the 100 cases. Otherwise the re-test stops for diagnosis. (4) **Methods:** (a) the predictor: batch size 1, a compiled `tf.function` call per case, pinned, timed as the median of 5 calls after 20 warm-up calls; (b) the top88 formulation with S-SuperLU (the typical Python port), S-PARDISO and S-GPU, each run to top88's stopping rule and timed setup-inclusive, with one warm-up and three repeats per case, reported as the median. (5) **Quality:** one evaluator scores every design: fem2d, E_min = 1e-9, p = 3, SuperLU. It applies to the predictor's continuous output (as the article evaluates it), the stored top88 designs and our designs. (6) **Primary:** the ratio of mean per-design times over the 100 cases (the article's statistic), comparing the strong baseline run to its stopping rule with the predictor. **Secondary:** matched quality, meaning per case the setup-inclusive time to the first iteration whose compliance is at most the predictor's; the OC update holds the volume exactly. A case that never matches is reported as not matched and ranked slowest. Also secondary: the typical Python port on our machine against the predictor, which repeats the article's comparison. (7) The article's values (1.9 s and 7.3 ms, 259×; the stored per-case top88 times average 1.896 s) are reported separately and labelled	The article's 259× compares a trained predictor with top88 on unstated hardware. The re-test puts both on one machine and adds the strong baseline

Date	Section	Change	Reason
2026-09-24	P00150 execution environment (before any P00150 predictor run; amends item 1 of the P00150 row)	The predictor runs in a separate Python virtual environment, created from the interpreter of conda env `Pytorch` (Python 3.10.18), with `tensorflow==2.15.1` from PyPI (CPU only on native Windows). The `Pytorch` env is not modified. Log: `retest/logs/tf_p00150_env_install_2026-09-24.log`	Creating a new conda env could require accepting the Anaconda channel terms, which needs the author's approval; a venv from an existing interpreter avoids that. TensorFlow 2.15 still loads SavedModels through `tf.keras`, as the harness expects
2026-09-24	P00150 reconstruction diagnosis (the pre-registered check stopped the re-test; recorded before the check is re-run)	Under the first mapping (v1: the whole left edge clamped; the input's y load taken as fem2d's +y), the untimed SuperLU scan gave compliance ratios (stored ÷ reproduced) of 0.55–1.49 over the first 11 cases, so the check failed and the re-test stopped. Diagnosis evaluated the article's own stored top88 designs with our finite-element model under candidate mappings (`retest/runs/p00150/_diagnosis/`). Two errors in our reconstruction were found. (1) The y load component follows a y-down frame, so it is negated in fem2d; this left ratios of 0.99–1.37 with a spread of ±8 %. (2) Only the 9 fine nodes that coincide with the clamped coarse nodes are supports, not the whole edge. With both fixes (mapping v2), 60 of 100 stored designs evaluate within 1 % of their stored compliance (many to 4 digits), and the other 40 evaluate 1–12 % above it. The misfits are bending-dominated, lower-volume cases, not yet explained; plane strain, a filtered field and E_min = 1e-3 were tested and make the fit worse or leave it unchanged. The pre-registered check (item 3: our top88 run to its stopping point against the stored compliance, ≥ 95 of 100 within 1 %) is now re-run under mapping v2. If it fails again, the P00150 speed comparison is not reported as a re-test; only the diagnosis is reported	The check did its job: the first reconstruction was wrong. Both fixes rest on the article's own data, the stored designs and compliances, not on any speed result
2026-09-25	P00150 reconstruction check, result (mapping v2)	Failed. Our top88 port matched the stored top88 compliance within 1 % at top88's stopping point on 57 of 100 cases (SuperLU port; the GPU solver gives the same 57, case for case); the pass condition was at least 95. In the other cases the stored compliance lies up to 19 % below our reproduction, mostly in bending-dominated cases. Per the diagnosis row, no P00150 speed comparison is reported; the manuscript reports the check and the diagnosis only (`evidence/p00150_check.json`, `retest/runs/p00150/_diagnosis/`). The predictor was never run, and the TensorFlow venv was removed as unused	Pre-registered stopping rule applied as written

2. Final check under the corrected mapping

Pass condition: >= 95 of 100 within 1 % (SuperLU port, stopping point). Result with mapping v2 (`p00150_check.json`): 57 of 100 cases within 1 % (SuperLU port; 0 runs without a stop), ratio stored ÷ ours from 0.814 to 0.9999, median 0.994. The GPU path gave 57 of 100. Passed: no. The remaining misfit is one-sided (our compliance at or above the stored value). The diagnosis located the misfits in bending-dominated, lower-volume cases; they remain unexplained.

Case	Stop iteration	Stored compliance	Ours	Stored ÷ ours
0	184	1.1625	1.3003	0.8940
1	66	0.085635	0.085651	0.9998
2	40	0.14083	0.14304	0.9846
3	228	0.30073	0.31644	0.9504
4	211	0.73341	0.81362	0.9014
5	172	0.49336	0.52253	0.9442
6	103	0.30185	0.32133	0.9394
7	237	0.3206	0.35435	0.9047
8	242	0.24865	0.25657	0.9692
9	83	0.26105	0.28638	0.9115
10	243	0.99061	1.1379	0.8706
11	93	0.19785	0.19798	0.9993
12	116	0.17494	0.175	0.9997
13	484	0.59007	0.67282	0.8770
14	144	0.1182	0.11827	0.9994
15	302	0.16188	0.16203	0.9991
16	112	0.21914	0.22357	0.9802
17	139	0.165	0.16511	0.9993
18	72	0.080916	0.080941	0.9997
19	111	0.16289	0.16326	0.9977
20	112	0.14448	0.14892	0.9702
21	72	0.086448	0.086458	0.9999
22	48	0.10799	0.10832	0.9969
23	98	0.083088	0.083169	0.9990
24	85	0.12486	0.12509	0.9982
25	204	0.46115	0.478	0.9647
26	250	0.20243	0.20473	0.9888
27	69	0.18941	0.19016	0.9961
28	225	0.38439	0.40606	0.9466

Case	Stop iteration	Stored compliance	Ours	Stored ÷ ours
29	117	0.091282	0.091361	0.9991
30	74	0.18746	0.1973	0.9501
31	99	0.24871	0.25682	0.9684
32	62	0.099073	0.099413	0.9966
33	54	0.28859	0.29846	0.9669
34	220	0.50214	0.58734	0.8549
35	70	0.17922	0.17965	0.9976
36	127	0.34448	0.36546	0.9426
37	116	0.083513	0.083519	0.9999
38	90	0.12188	0.12208	0.9983
39	75	0.18614	0.18638	0.9987
40	284	0.99897	1.2275	0.8138
41	129	0.13621	0.13702	0.9941
42	115	0.11137	0.1119	0.9952
43	85	0.12173	0.12194	0.9982
44	65	0.18737	0.18765	0.9985
45	321	0.39138	0.40279	0.9717
46	137	0.31092	0.32949	0.9436
47	205	0.22049	0.22253	0.9908
48	65	0.087633	0.087679	0.9995
49	89	0.13942	0.13965	0.9984
50	84	0.15266	0.15321	0.9964
51	235	0.57786	0.59552	0.9703
52	200	0.10713	0.10723	0.9991
53	44	0.19937	0.1994	0.9999
54	60	0.29912	0.30492	0.9810
55	57	0.29501	0.29667	0.9944
56	68	0.099156	0.099186	0.9997
57	222	0.68185	0.72705	0.9378
58	59	0.18368	0.18374	0.9997
59	258	0.22734	0.22969	0.9898
60	81	0.12044	0.12102	0.9952
61	156	0.10481	0.10487	0.9994
62	144	0.14923	0.15078	0.9897
63	122	0.76419	0.79638	0.9596
64	77	0.11261	0.113	0.9966
65	65	0.21343	0.2139	0.9978
66	296	0.092293	0.092322	0.9997
67	37	0.21291	0.21474	0.9915
68	91	0.32833	0.34363	0.9555
69	270	0.24314	0.2458	0.9892
70	67	0.093721	0.093862	0.9985
71	100	0.11955	0.11977	0.9982
72	54	0.19905	0.19909	0.9998
73	84	0.23268	0.2336	0.9960
74	149	0.094671	0.094719	0.9995
75	204	0.43898	0.44582	0.9847
76	71	0.33415	0.34796	0.9603
77	120	0.20797	0.20864	0.9968
78	79	0.33015	0.354	0.9326
79	255	0.22265	0.2244	0.9922
80	46	0.091836	0.091852	0.9998
81	74	0.14384	0.14404	0.9986
82	153	0.18054	0.18977	0.9514
83	92	0.51405	0.54645	0.9407
84	64	0.1523	0.15271	0.9973
85	194	0.11963	0.11986	0.9981
86	160	0.63957	0.6977	0.9167
87	341	0.22524	0.23108	0.9747
88	61	0.18787	0.18825	0.9980
89	116	0.13224	0.13277	0.9960
90	48	0.1027	0.10272	0.9998
91	78	0.13276	0.13873	0.9569
92	57	0.20332	0.20444	0.9945
93	140	0.45126	0.45914	0.9828
94	259	0.75011	0.87527	0.8570
95	73	0.085913	0.086619	0.9919
96	147	0.20965	0.21081	0.9945
97	183	0.31785	0.32542	0.9767
98	39	0.095393	0.096225	0.9914
99	71	0.084622	0.084638	0.9998

3. Scripts

1. `retest/p00150_harness/check.py`: P00150 reconstruction check (plan/BASELINE_PREREG.md §9, P00150 row item 3; mapping v2 after the diagnosis row).
2. `retest/p00150_harness/classical.py`: Classical top88-formulation runs on the 100 P00150 validation problems (plan/BASELINE_PREREG.md §9, P00150 row).
3. `retest/p00150_harness/evaluate.py`: P00150 re-test scoring: image orientation, reconstruction check, compliance of every design by one evaluator.

4. `retest/p00150_harness/problems.py`: The 100 validation problems of P00150 (Halle et al., Appl. Sci. 2021; Mendeley data, CC BY 4.0) as fem2d Problems.

5. `retest/p00150_harness/run_pen.py`: P00150 predictor (model_predictor_64x64, Mendeley data, CC BY 4.0) on the 100 validation inputs, timed per design.